

Báo cáo rà soát test code

<!-- framework: Jest | run_id: run_20260620_032856_634252 -->

Framework: Jest | Run ID: run_20260620_032856_634252

Tóm tắt kỹ thuật

- Framework: Jest
- Kết quả chạy: Tất cả 13 test được liệt kê trong file generated.test.js đều PASS.
- Coverage: 90 % (đạt ngưỡng coverage).
- Issue type: Không có lỗi chạy, không có vấn đề về công cụ.
- Blocker quan trọng nhất: Không có lỗi nghiêm trọng ngăn test chạy; tuy nhiên hai test liên quan tới roundMoney (TC-003, TC-004) không được thực thi vì chúng không ràng buộc với bất kỳ hàm xuất khẩu nào, dẫn đến thiếu bao phủ cho hàm roundMoney.

Artifacts được xem xét:

- Mã nguồn source_under_test.js (từ orderPricing.js).
- Test tự động sinh generated.test.js.
- Kế hoạch test (Test Plan JSON) và bằng chứng thực thi (Execution evidence).

Lỗi nghiêm trọng

- Không phát hiện.

Test còn thiếu

- TC-003 & TC-004: Hai test cho hàm roundMoney được đánh dấu *ambiguous* trong Test Plan và không có trong file test sinh ra, vì không có export nào trùng khớp. Điều này để lại khoảng trống kiểm thử cho hàm roundMoney, mặc dù hàm này được export và được sử dụng trong các hàm khác.
- Bằng chứng: source_under_test.js xuất khẩu roundMoney nhưng generated.test.js không có test nào gọi target.roundMoney.
- Ảnh hưởng: Không kiểm chứng được tính năng làm tròn đúng (đặc biệt là trường hợp .005) → rủi ro lỗi tính toán tiền.
- Cách sửa: Thêm hai test tương tự TC-003 và TC-004, ví dụ:

```
test('TC-003 roundMoney rounds typical decimal value up', () => {  
  expect(target.roundMoney(12.3456)).toBe(12.35);  
});  
test('TC-004 roundMoney rounds half-up at .005', () => {  
  expect(target.roundMoney(7.125)).toBe(7.13);  
});
```
- Kiểm thử cho finalTotal không kiểm chứng việc gọi roundMoney nội bộ: Mặc dù yêu cầu “finalTotal should use roundMoney for rounding”, không có assertion nào xác nhận rằng hàm roundMoney được gọi.
- Bằng chứng: Các test TC-012 và TC-013 chỉ kiểm tra giá trị trả về, không mock/spy roundMoney.
- Ảnh hưởng: Nếu finalTotal thay đổi cách làm tròn, các test hiện tại vẫn có thể PASS.
- Cách sửa: Sử dụng jest.spyOn(target, 'roundMoney') để xác nhận rằng hàm được gọi một

lần với giá trị tính toán đúng.

Assertion yếu

- Không phát hiện. (Tất cả các test có assertion cụ thể `toBe` hoặc `toThrowError`.)

Rủi ro maintainability

- Mocking trực tiếp `shippingFee` trong TC-012 và TC-013: Việc mock hàm nội bộ bằng `jest.spyOn` là hợp lý, nhưng nếu trong tương lai `finalTotal` thay đổi cách gọi (ví dụ chuyển sang import khác), các mock này có thể không còn hiệu lực, gây flaky tests.
- Cách giảm rủi ro: Đặt mock trong `beforeEach/afterEach` và luôn `mockRestore()`; hoặc tách `shippingFee` ra module riêng để dễ mock.
- Không có test cho trường hợp `customerTier` không phải chuỗi: Yêu cầu không rõ ràng, nhưng nếu hàm `calculateDiscount` nhận giá trị không phải chuỗi, hiện tại không có kiểm thử bảo vệ. Đây là rủi ro tiềm ẩn.

Gợi ý sửa ngay

1. Thêm test cho `roundMoney` (TC-003, TC-004).
2. Thêm spy/assertion để xác nhận `finalTotal` gọi `roundMoney`.
3. Cấu trúc lại mock `shippingFee` thành `beforeEach/afterEach` để tránh quên `mockRestore`.
4. (Tùy chọn) Thêm test cho `calculateDiscount` với `customerTier` không phải chuỗi để bảo vệ trước lỗi không mong muốn.
